

PaxosLease Revisited: A Checked Model of Diskless Distributed Leases

Márton Trencsényi (mtrencseni@gmail.com)

Abstract

PaxosLease is a protocol by which a quorum of acceptors grants time-bounded exclusive ownership with no durable acceptor lease state and no disk write on the lease acquisition path. This paper gives a precise, machine-checked statement of the protocol and of its standard use, electing a Multi-Paxos leader that recovers once per epoch and then commits with single-round appends. Three results are new. First, the restart quarantine an acceptor must observe after losing its volatile state is the proposer attempt duration, a bound tight in the checked model and machine-checked at the boundary in both directions. Second, timer placement is safety-critical: starting the attempt timer at prepare-quorum receipt can produce two simultaneous lease owners. Third, when a proposer abandons an acquisition attempt and retries, messages left over from the abandoned attempt can make acceptors report a lease the proposer itself appears to own; unless such a report counts as open only while the proposer is renewing that exact lease, two simultaneous owners follow at any quarantine length. The lease transition system is formalized in TLA+ and checked by TLC, and the timing arithmetic is proved in TLAPS. An executable reference model encodes the same rules a second time, independently of the TLA+, so that a wrong rule has to survive two separate encodings. A single-file runnable demonstration gives the algorithm in readable Python code.

1. Introduction

PaxosLease [1] is a protocol by which a quorum of acceptors grants time-bounded exclusive ownership, diskless in the sense that acceptors keep no durable lease state and the lease acquisition path writes nothing to disk. This paper gives a precise, machine-checked statement of the protocol and of its standard use, electing a Multi-Paxos leader that recovers once per epoch and then commits with single-round appends.

A lease is a lock whose ownership is bounded by time [3]: a process may act as owner only until its lease expires, and a process that granted the lease refuses conflicting grants until its own exclusion interval has expired. Exclusion is the property that at any real instant at most one process has the authority to act as owner. The time bound is what makes the lock survivable in a distributed system, where the remaining processes cannot tell a failed owner from a slow one, and it is also what makes exclusion delicate, since exclusion then depends not only on messages and quorums but on assumptions about elapsed time. Clocks need not be synchronized, but the intervals measured by different processes must have bounded error. PaxosLease uses ballots and intersecting quorums, as Paxos does, to grant such a time-bounded leadership right; it does not replace Paxos's own recovery (Section 5).

This paper makes three contributions. First, the tight restart bound: the quarantine an acceptor must observe after losing its volatile state is the proposer attempt duration D_P , tight in the checked model, checked at the boundary in both directions, smaller than the acceptor exclusion duration and smaller than the maximal lease time (Sec-

tion 7). Second, two validity conditions on attempt evidence, each backed by a machine-found safety-violating two-owner execution. The first is the attempt time bound, whose placement admits two unsafe variants, the second of which is what both audited implementations ship. The second is the renewal qualifier: a proposer may treat a report of its own lease as an open response only while it is renewing that exact lease, a condition the model checker itself surfaced once retry was modeled explicitly (Section 8). Third, the implementation audit: eight source-level findings in Keyspace and ScalienDB read against the checked model, backed by checked projections of the shipped protocol and by executable witnesses (Section 11, Appendix C). Supporting these are a statement of the algorithm precise enough to resolve the internal inconsistency of the previous publication, each step bound to the TLA+ action that formalizes it, with the checked module normative (Section 4); the leased-leader Paxos lifecycle stated in the same discipline (Section 5); and a repository whose negative claims are machine-checked rather than asserted: mechanically generated one-rule-weakened variants whose violations TLC must find on every run, fifteen structured witnesses, an executable reference model that encodes the rules a second time, a fail-fast evidence pipeline, and a single-file runnable demonstration (Sections 9 and 10).

The 2012 publication states the safety-critical timer rule inconsistently, one way in its figure and another in its pseudocode. The two production implementations, Keyspace [14] and ScalienDB [15], copied the pseudocode, were protected by unintended implementation choices, but are defeated by ordinary timer-dispatch latency. Section 11

presents the findings, [Section 12](#) draws the lessons, and [Appendix B](#) and [Appendix C](#) give the source-level detail. This paper supersedes the 2012 publication: [Section 2](#) gives the protocol in plain terms and [Section 4](#) gives it precisely, and a reader who wants to implement PaxosLease should work from this new source and not from [1].

The rest of the paper is organized as follows. [Section 2](#) describes the protocol in plain terms, [Section 3](#) states the system model, [Section 4](#) states the algorithm, and [Section 5](#) states the leased-leader algorithm. [Section 6](#) gives the safety argument, [Section 7](#) establishes the quarantine bound, and [Section 8](#) gives the two rules whose plausible variants are unsafe. [Section 9](#) describes the checked models and tests, [Section 10](#) presents the runnable demonstration, [Section 11](#) audits the unformalized years, [Section 12](#) draws the lessons, and [Section 13](#) discusses related work. [Section 14](#) lists limitations. [Appendix A](#) records the checked invariants, [Appendix B](#) details the inconsistency in the original statement, and [Appendix C](#) details the implementation audit, [Appendix D](#) inventories the evidence, and [Appendix E](#) records how the repository was built.

2. How PaxosLease Works

PaxosLease grants one process the exclusive right to act as owner of something, for a bounded stretch of time, using a set of machines any of which may crash and none of which writes to disk in order to grant it. This section describes how it does that in plain terms. It is intended for a reader who has not seen the protocol before and who should not have to reconstruct it from the numbered steps of [Section 4](#).

The reason disklessness is the interesting constraint is that the obvious way to grant an exclusive right is to write down who has it. A grantor that has written nothing down and then restarts remembers nothing, and a grantor that remembers nothing will happily grant the same right twice. The protocol must therefore prevent a restarted grantor from issuing a conflicting grant while any lease or acquisition attempt that depended on its forgotten state may still be effective.

The idea is to run Paxos, with one substitution: the value being agreed upon is not a log entry but a statement of the form “node 3 owns the lease, for the next seven seconds.” Paxos guarantees that a quorum of acceptors agrees on at most one value, which is exactly the guarantee a lease needs. Unlike an ordinary Paxos value, a lease has a bounded validity interval. An acceptor does not need to remember a lease forever, only until it expires, and that bounded lifetime is what makes it safe for the acceptors to keep the whole thing in memory.

An acquisition is two rounds. A proposer picks a ballot number that nobody has used before and sends it to every acceptor, asking two things at once: to promise not to entertain any smaller ballot from now on, and to respond whether it is currently holding a live lease for another owner. If a majority of acceptors answer, and none

of them reports a live lease belonging to another owner, the proposer sends a second message asking those acceptors to record it as the owner for a fixed duration. When a majority confirm, the proposer is the owner, and it informs the learners.

Majorities are what make this work, because any two majorities of the same set share at least one member. A second proposer trying to acquire the lease cannot avoid talking to at least one acceptor that already answered the first, and that acceptor either reports the live lease, which stops the second proposer, or has promised a ballot that makes the first proposer’s remaining messages ineffective. There is no configuration of messages in which two proposers each collect a majority in ignorance of the other.

What Paxos does not have to reason about, and this protocol does, is time. Two obligations follow, and they are the subject of much of this paper. The first is that an owner’s authority must end before the acceptors’ exclusion does. The acceptors are holding the lease open for a fixed duration measured on their own clocks, and if the owner believes it holds the lease for even slightly longer than the acceptors are willing to enforce it, then in that gap a second proposer can acquire while the first still believes itself the owner. The second is about forgetting: An acceptor that crashes loses its promises and its record of the live lease, and if it restarts and immediately starts answering again, it can help a second proposer acquire a lease that, as far as everybody else is concerned, is already held. The repair is a quarantine: a restarted acceptor refuses all lease-layer messages for a fixed interval, and the design question is how long that interval must be.

Two smaller pieces complete the picture. A lease is renewed by running the same two rounds again, under a higher ballot, before the current one expires; a renewal that fails to complete extends nothing, and the owner’s authority simply runs out. A lease may also be released early, which is a courtesy that speeds up the next acquisition and is never required for safety.

Everything after this section is either a precise statement of it, a proof obligation about it, or an account of the ways it can be got wrong. The precise statement is [Section 4](#), whose numbered steps are bound to a machine-checked TLA+ module.

3. System Model

The system contains a finite set of proposers and a finite set of acceptors. A quorum system is a collection of sets of acceptors, any two of which intersect; the executable models use majorities.

Processes are not Byzantine, but any process may crash. A crashed proposer loses its lease authority, and a crashed acceptor loses its PaxosLease state, including its promised ballot and its accepted lease. This is the sense in which the protocol is diskless. When PaxosLease is used with Paxos, the volatile loss does not extend to the Paxos log, whose acceptor state remains durable.

One durability caveat applies to proposers. Ballots must be globally unique, including across restarts of the same proposer. Uniqueness is obtained from a restart counter written to stable storage [1]. (A sufficiently large random component gives probabilistic rather than deterministic uniqueness, and a deployment that relies on it should say so.) The counter is one word written per proposer restart, not a write on the protocol path, but a fully diskless deployment must obtain uniqueness in some other way. Flease [10], for example, derives ballots from loosely synchronized clocks (Section 13). The models make uniqueness structural: the TLA+ specification tracks used ballots in a global history set, and the Python models use the counter-major (*counter, restart, node*) layout of Section 4, so ballots remain comparable across proposers. Note that the restart component provides uniqueness across a crash, not ordering, since the counter restarts under a higher restart component and $(1, restart+1, node) < (100, restart, node)$.

Messages may be delayed, reordered, duplicated, or lost. A message carries a ballot and, in an accept request, the identity of the proposed lease instance. Messages do not carry synchronized timestamps. Each process uses its own elapsed-time measurements to decide whether a lease or a quarantine interval has expired. Messages are not tied to connections: unless stated otherwise, a message sent before its addressee crashed may be delivered after the restart. This is the default in the models, which also test the connection-oriented alternative, in which a crash discards the messages addressed to the crashed process; the results hold under both (Sections 8 and 9).

The timing condition needed for safety is a containment condition. Whenever acceptor *a* accepts a lease for proposer *p*, the end of *p*'s usable interval must come no later than the end of *a*'s exclusion interval. If all clocks run at the same rate and the proposer starts its timer before the acceptor starts its own, then equal local durations suffice. If clock rates can differ by at most ρ , with rate bounds $r_{min} = 1 - \rho$ and $r_{max} = 1 + \rho$, then a conservative choice is

$$D_P \cdot r_{max} \leq D_A \cdot r_{min},$$

where D_P is the proposer's local duration and D_A is the acceptor's local exclusion duration. The condition covers the worst case, in which the proposer's clock runs slow and stretches D_P out in real time while the acceptor's runs fast and uses D_A up early. The arithmetic content of this condition, that bounded rates together with the cross-multiplied duration inequality imply real-time containment, is proved in TLAPS as `DriftContainmentArithmetic`.

The symbols c, β, γ, k and τ are deliberately left unused here, so that a companion treatment of participants in relative motion can take them with their standard relativistic meanings. When the proposer's timer starts is part of the protocol, not a detail of implementation. The model starts the pending deadline when `Prepare` is sent. Section 8 shows that the alternative, starting it at prepare-quorum receipt, is unsafe when promises are volatile, and Section 7

shows that the prepare-time rule is also what makes a small quarantine possible.

4. PaxosLease Algorithm

We state the algorithm as numbered steps. Every step names, in brackets, the action of `tla/spec/PaxosLease.tla` that formalizes it. The TLA+ module, not this prose, is the normative statement of the PaxosLease algorithm.

4.1. State

Proposer state is per attempt and volatile, with one durable exception.

- the current ballot b , and the phase of the attempt (idle, preparing, accepting);
- the pending deadline t_d of the attempt, assigned when `Prepare` is broadcast (T1) and stored, not scheduled;
- the renewal base: the ballot of the lease this attempt renews, or none if the attempt is a fresh acquisition (P1, P2);
- the identities of the acceptors that have responded in the current attempt, kept as a set rather than a count (P2);
- when active, the authority deadline and the quorum certificate that established the lease;
- the restart counter, which is the one piece of proposer state written to stable storage (Section 3).

Acceptor state is entirely volatile, which is what makes the protocol diskless and what the restart quarantine exists to repair.

- the highest ballot promised since the last restart;
- the accepted lease instance, if one exists;
- the local deadline until which that lease excludes conflicting grants;
- after a restart, the deadline until which the acceptor refuses all lease-layer messages (A4, T3).

A lease instance is identified by (*owner, ballot*). Ballots are globally unique, so the owner is defensive redundancy, but the pair is what release must match (A3).

4.2. Proposer

P1 [StartAcquire]: Choose a fresh ballot b , never used by any proposer, including this proposer's own earlier incarnations: counter-major (*counter, restart, node*), that is, ballots compare lexicographically with *counter* most significant and *node* as the final tiebreak, where *counter* increments per attempt and *restart* is a durably stored counter incremented at every process start, so no ballot is reused

across a crash. Record as the attempt's *renewal base* the ballot of the lease this proposer currently holds, or none if this is a fresh acquisition; P2 tests self-owned reports against it. Assign the pending deadline $t_d := \text{now} + D_P$ as *stored state*, and broadcast `Prepare(b)`.

P2 [DeliverPromise]: On a promise for the current b : if it reports no live lease, record the responder's *identity* and call the response *open*. For a renewal attempt, a response is also open, but only when it reports the same owner *and the same ballot* as the active lease captured at P1 as the attempt's renewal base. Quorums are sets of distinct acceptors, not counts of messages (Section 11, fourth finding). The renewal clause checks where the reported lease came from, not only who owns it: the attempt records at P1 which lease it is renewing, and P2 admits a self-owned report only if it names that exact lease. A self-owned record under any other ballot may have been installed by an accept request an abandoned attempt left in flight, and it blocks like a foreign lease (Section 8).

P3 [SendAccept]: When promises from a quorum are recorded and $\text{now} < t_d$: broadcast `Accept(b, self)`.

P4 [DeliverAccepted]: On accepted responses for b from a quorum of distinct acceptors, and only if $\text{now} < t_d$, checked against the stored deadline inside this transition rather than by any timer callback: become active until t_d , record the certificate, and broadcast `LearnChosen(self, d)`.

P5 [AbandonAttempt, then P1]: An attempt that cannot proceed is abandoned: the proposer returns to idle, alive and keeping whatever lease it already holds, and retries with a fresh, higher ballot from P1. Messages of the abandoned attempt may still be in flight, and the ballot comparison in P2 and P4 discards every response belonging to it. Retry does not advance the durable restart counter; only a crash and restart does.

P6 [Release, optional]: To release, first stop acting as owner, then broadcast `Release(b)` naming the exact instance. Release is an availability optimization, never a safety requirement.

Renewal is a new acquisition by the same owner under a higher ballot, entered at P1, whose renewal base is then the ballot of the lease being extended. The old lease remains the only authority until the new accept quorum is complete, and a failed renewal extends nothing. Capturing the base at P1 rather than inferring it at each response means a fresh attempt can never be reclassified as a renewal by ownership changing while responses arrive.

4.3. Acceptor

A1 [DeliverPrepare]: On `Prepare(b)`: if $b \geq$ the promised ballot, set $\text{promised} := b$ and reply with the currently live accepted lease, reporting an expired lease as no lease; otherwise reply rejecting, or not at all. A rejection is a liveness aid, and to serve T4 it should carry the promised ballot. Exclusion does not depend on rejections at all: an

acceptor that stays silent refuses just as effectively, and the proposer's own deadline ends the attempt either way.

A2 [DeliverAcceptReq]: On `Accept(b, o)`: if $b \geq$ promised, set $\text{promised} := b$, record the lease (o, b) with exclusion deadline $\text{now} + D_A$, and reply accepted; otherwise reject.

A3 [DeliverRelease]: On `Release` from o with ballot b : clear the accepted lease only if it is exactly (o, b) . Matching on owner alone erases a later renewal (Section 9, 36-state trace).

A4 [RestartAcceptor]: On restart after losing volatile state: refuse every lease-layer message, prepare, accept, and release alike, until $\text{now} + \text{Quarantine}$.

4.4. Learner

L1: Activation is made application-visible through learners. The owner's own learner installs the absolute deadline t_d ; a remote learner may cache the owner with a conservative local expiry, but only the owner acts on the lease. A remote learner that measures the duration from the moment the message arrives, rather than from the moment the lease began, is assuming a bound on delivery delay, and that bound must then be stated (Section 11, sixth finding).

4.5. Timing rules

The timing rules are stated as named rules, not as properties implied by the steps.

T1 (attempt time bound): The pending deadline t_d is assigned when `Prepare` is broadcast (P1) and checked in the transitions that use the attempt's evidence (P3, P4). It is data, not a scheduled callback: a paused process loses its dispatch order, and the deadline must not be lost with it (Section 11, third finding).

T2 (containment): $D_P \leq D_A$, so an active owner's authority interval is contained in every certificate acceptor's exclusion interval. With clock-rate error bounds $r_{\min}, r_{\max}$ the condition is $D_P \cdot r_{\max} \leq D_A \cdot r_{\min}$ (Section 3).

T3 (quarantine):

$$\text{Quarantine} \geq D_P,$$

rate-adjusted like T2. It is not the acceptor exclusion duration and it is not the maximal lease time; Section 7 derives the bound, shows by exhaustive checking that it suffices even when $D_P < D_A$, and shows that it is tight.

T4 (ballot hygiene): Safety requires exactly one property: ballots are globally unique and never reused across restarts (the durable *restart* component of P1). The rest of the rule is liveness. Because uniqueness does not order a restarted proposer's ballots above its pre-crash promises (Section 3), a proposer raises its counter above any ballot it observes in responses, which in turn requires rejections that carry the promised ballot (A1).

The model has a variable *now*, and executable guards such as $\text{now} < \text{pendingDeadline}$ read it directly. Since

this can look like an assumption of synchronized clocks, we state precisely what it means. The variable `now` is the common real-time axis on which safety is asserted, and each local timer is a duration that has been conservatively translated onto that axis using the rate bounds of T2. When a process in the model compares `now` with its own deadline, it represents an implementation reading its own monotonic elapsed-time counter, never another process’s clock. Similarly, the certificate variables `cert` and `certDeadline` that appear in the invariants are history variables. They record, for the benefit of the checked predicates, which acceptors granted the current lease and the exclusion deadlines those acceptors reported. The protocol itself never compares a deadline measured on one clock with a deadline measured on another.

5. Leased-Leader Algorithm

PaxosLease is not Paxos, and the two should not be confused. Paxos solves agreement for log slots, choosing at most one value in each slot [2]. Unlike the lease layer, its acceptors are durable: a promise and an accepted value must each reach stable storage before the acceptor replies, so committing one value in plain Paxos costs two round trips and two durable writes at every acceptor.

The leader optimization removes half of that. A proposer runs Phase 1 once under a single ballot for the whole log rather than for one slot, and the promises it collects reserve that ballot for every slot it will later fill. From then on it appends by running Phase 2 alone, so each new value costs one round trip and one durable write per acceptor instead of two. Phase 1 is amortized, not skipped, and the reservation lasts until another proposer’s own Phase 1 wins a quorum of promises under a higher ballot.

This is what the lease contributes. A proposer holding a PaxosLease renews it, and may go on renewing it indefinitely: until it crashes, until the network prevents a renewal, or until the application hands leadership elsewhere. While it holds the lease no other proposer can acquire the lease, and hence no other proposer will run a Prepare phase, so the single reservation stays valid across arbitrarily many appends and the two round trips of the initial full round are amortized over all of them. Driven to the limit, the amortized cost of committing a value is one round trip and one durable write at each acceptor, which is what makes leased Multi-Paxos a practical way to implement a replicated log.

What the lease does not do is replace Phase 1. A proposer that has just acquired the lease does not know which values earlier leaders may already have had accepted, or chosen, and it must run a full first round to discover them and finish them before proposing anything of its own.

The standard use of the lease, and the one Keyspace and ScalienDB implement, is this classic Multi-Paxos leader optimization. We state the leader lifecycle precisely, in the discipline of [Section 4](#): each

step names its counterpart in the composition model (`tla/spec/PaxosLeasePaxos.tla`).

M1 [AcquireLease]: Win the PaxosLease of [Section 4](#). This starts a fresh *leadership epoch*, the interval over which one proposer holds the lease continuously, across any number of renewals, and it *clears* any recovery standing from an earlier epoch.

M2 [StartRecovery/FinishRecovery]: Run a log-wide Paxos Phase 1 under a fresh ballot. In the Multi-Paxos organization assumed here, and implemented by both audited systems, a promise quantifies over all instances rather than one [2, 9], so the single round both reserves the ballot for the entire log and discovers the accepted values that constrain recovery.

M3 [repair]: For every slot that Phase 1 reports constrained, select the value attached to the highest-ballot accepted response for that slot, re-propose it under the reserved ballot, and learn the results. Phase 1 alone is not cleanup: a value that is discovered but not re-proposed leaves recovery to be finished by whatever proposal next touches the slot.

M4 [BecomeReady]: Only now is the leader ready. Readiness is per epoch: the invariant `ReadyImpliesRecovered` checks that a re-elected leader repeats M2 and M3 rather than relying on a previous epoch’s recovery.

M5 [AdmitOrApply]: Fast mode. Each client command is placed, unaltered, into a new slot with a single accept round under the reserved ballot and no per-instance Prepare.

M1 relies on the exclusion theorem that [Section 6](#) establishes for the lease layer; none of M2 through M5 does. Paxos agreement does not depend on lease exclusivity. Acceptors reject lower-ballot accepts regardless of any clock, so even two processes that simultaneously believe themselves leader cannot choose conflicting values, and skipping Prepare in M5 is sound because M2 already promised the whole log. The lease adds no safety to this and is needed for none of it. What the lease adds is efficiency and liveness, since with at most one real-time leader the fast path is not repeatedly preempted by dueling ballots, and local admission and lease-guarded reads become possible. One caution on reads: the lease alone does not make a local read current. A leader serving reads locally needs the lease, this epoch’s completed recovery (M2, M3), and an applied state machine covering the relevant prefix; ownership without recovery reads stale state. Conversely, the fast mode imposes only one obligation on the lease layer: no admission before the leader’s own epoch has completed M3.

The PaxosLease layer and the Paxos layer have a one-way interface, which is what makes them independently reviewable. The PaxosLease layer determines which Paxos proposer may run M2 through M5, and when it may begin. No Paxos transition reads PaxosLease state: a Paxos acceptor deciding whether to promise or to accept consults its own promised ballot and its own accepted values, and

nothing else, and it does not know whether the Paxos proposer addressing it holds the lease. No Paxos transition reads a clock: deadlines, durations, drift bounds and the quarantine all live in the PaxosLease layer, and agreement on the log is a function of the message history alone. The optimization therefore changes only the Paxos proposer, which skips the prepare phase while the lease condition holds; the Paxos acceptor and the Paxos learner are unmodified.

6. Safety Argument

Theorem. At any time, at most one proposer is active.

The theorem is the invariant checked in `tla/spec/PaxosLease.tla`:

```
LeaseExclusivity == Cardinality(active) <= 1
```

where `active` is the set of proposers that currently hold lease authority. Every configuration of the lease module checks it, and the counterexample variants exist to violate it.

What follows is an argument rather than a mechanized proof of the transition system. TLC establishes the invariant exhaustively for the finite configurations it is given, and no further; [Section 14](#) states what that leaves open. The arithmetic the argument relies on is proved mechanically in TLAPS: timer containment, the coverage of forgotten facts by the quarantine, and the clock-rate condition of [Section 3](#).

Argument. A proposer is active only if it holds a certificate from a quorum of acceptors. Suppose that proposer p is active with certificate quorum Q_p and that a different proposer q later becomes active. Before q can become active it must obtain a prepare quorum Q_q , and since quorums intersect, some acceptor belongs to both Q_p and Q_q .

If that acceptor still holds a live accepted lease for p when it answers q 's prepare, then q cannot proceed with a conflicting lease. If it answers with no live lease, then its exclusion interval for p has expired, and by timer containment p 's usable interval has expired as well, so p is no longer active. The remaining cases are lower-ballot races and acceptor crashes. Monotone promises dispose of lower-ballot races while an acceptor's volatile state persists; quarantine and the attempt time bound dispose of state loss across a restart. Both are necessary, and [Sections 7](#) and [8](#) show what fails when either is weakened.

The intersection step needs one further clause. The record at the intersecting acceptor is not the earlier owner's authority. It is only that acceptor's in-memory record of having granted it, and the two can come apart: a delayed accept request from an abandoned attempt can overwrite the record while the owner goes on holding the lease, unaware that anything at the acceptor has changed. A later acquirer must therefore consider where a reported lease came from. A fresh acquisition may count only responses

that report no live lease, and a response reporting the acquirer's own lease is open only for a renewal: an attempt begun while the acquirer is active, and only if the reported instance is the exact lease being renewed. Under that rule, an overwritten record can mislead no one, since it either names a foreign owner, which blocks, or names the acquirer under a ballot that is not its renewal base, which blocks equally. The TLA+ module tests the proposer's activity and active ballot at response delivery. With a single attempt in flight and activation updating atomically, that coincides with the renewal base captured at P1 when the attempt begins. An implementation should store the captured form, as the executable models do, because separate callbacks and delayed learners can otherwise reclassify an attempt while its responses arrive. An alternative places the clause at the acceptor instead, as an invariant of the grantor: an acceptor never replaces a live lease of a different owner. [Section 8](#) exhibits the two-owner execution that follows when the clause is dropped, and compares the two repairs.

The exclusion argument rests on the invariants listed in [Appendix A](#), of which the most important is timer containment: for every active proposer, the proposer's deadline comes no later than the exclusion deadline recorded by each acceptor in its certificate. Ballots and quorum intersection ensure that a later acquisition sees the earlier one, and timer containment explains why an acceptor that no longer sees the earlier lease is not concealing a still-active owner.

This is a safety argument only. It does not assert that any proposer eventually obtains a lease. Liveness requires eventual message delivery, sufficiently accurate timers, and a period during which a quorum of acceptors stays available. As in Paxos, two proposers can also starve one another with ever higher ballots, which implementations prevent with randomized backoff [\[1\]](#).

7. Restart Quarantine Bound

A restarted acceptor has forgotten two kinds of facts, promises and accepted leases, and we must determine how long it should stay silent so that neither can matter. The answer turns out to be the same for both.

1. A forgotten promise belongs to an attempt whose Prepare was sent at some time t_{prep} no later than the crash. Under the prepare-time timer rule, that attempt cannot activate after $t_{prep} + D_p$.
2. A forgotten accepted lease appears to be the harder case, because the acceptor promised exclusion for $D_A \geq D_p$, which suggests the bound $\max(D_p, D_A)$. That bound is too conservative, and the gap is instructive. Once the acceptor has forgotten the exclusion, the safety-relevant question is not whether the old exclusion interval has ended. It is whether any proposer that relied on the exclusion can still be, or can still

become, active. The lease the acceptor forgot was accepted at some t_{acc} no later than the crash, on behalf of an attempt whose Prepare was sent at $t_{prep} \leq t_{acc}$, so the proposer it supports is not active past $t_{prep} + D_P$, regardless of D_A .

Both forgotten facts therefore reduce to a single statement: everything an acceptor can forget belongs to an attempt that expires D_P after a Prepare that preceded the crash. That statement is the TLAPS obligation `QuarantineCoversForgottenFacts`. Hence, in the discrete model,

$$Quarantine \geq D_P$$

suffices, and the acceptor exclusion duration does not appear in the bound at all.

The bound above is stated in the idealized time of the model, where every clock advances at the same rate. Real clocks do not, and the adjustment is the same one [Section 3](#) applies to containment. The attempt is measured on the proposer’s clock, which may run slow and stretch D_P out in real time; the quarantine is measured on the restarted acceptor’s clock, which may run fast and use its interval up early. Taking the worst of both with the rate bounds $r_{min} = 1 - \rho$ and $r_{max} = 1 + \rho$ gives

$$Quarantine \cdot r_{min} \geq D_P \cdot r_{max},$$

which is the containment condition $D_P \cdot r_{max} \leq D_A \cdot r_{min}$ turned around: the same pair of rate bounds deflates the acceptor duration you may rely on and inflates the quarantine you must serve. Both follow from the single TLAPS obligation `DriftContainmentArithmetic`, once with the acceptor duration and once with the quarantine in its place, and the Python timing module computes both bounds this way.

The bound is the tight worst-case bound under the protocol and failure model stated here, established by the arithmetic lemmas together with finite model checking rather than by a parameterized proof. What tight means is that both directions are machine-checked at the boundary: quarantine D_P passes and quarantine $D_P - 1$ does not. [Section 14](#) states where that leaves the claim.

We checked it in both directions at two separated duration pairs, on the unmodified specification, with two proposers racing across acceptor crash and restart: quarantine D_P passes exhaustively and quarantine $D_P - 1$ yields a two-owner trace at each pair. [Appendix D](#) lists the configurations and their state counts.

8. Timer Placement and the Renewal Qualifier

Two rules of [Section 4](#) have plausible variants that are unsafe: where the attempt timer starts (T1), and how a proposer reads a prepare response that reports its own lease (P2). Neither two-owner execution below needs a lost message or a clock error.

T1: the attempt deadline starts when Prepare is sent, and it is stored state. Acceptor state is volatile, so evidence gathered before a crash must expire at a fixed time after it was created. T1 assigns t_d at P1 and checks it at P3 and P4, so the whole attempt, its unprocessed promises included, expires with it.

The pseudocode of [1] starts the timer at prepare-quorum receipt instead. The attempt then has no timer until the quorum arrives, a prepare quorum stays usable for an unbounded time, and no finite quarantine covers an attempt that has no deadline. TLC finds the two-owner execution at the full quarantine bound in 27 states with two acceptors, and in 25 states with three acceptors and a single crashed acceptor, the one in the intersection of the two proposers’ quorums. [Appendix B](#) walks the execution through and gives the failure in implementation terms.

The deadline must therefore be data that every handler using the attempt’s evidence checks. Keyspace and ScalienDB instead bound the attempt with a retry timeout, whose deadline exists only as dispatch order: a scheduling pause runs an expired attempt’s handlers before the timeout fires, and TLC finds the two-owner execution under the shipped constants ([Section 11](#), third finding). Durable promises and Fleas’s clock-derived ballots ([Section 13](#)) also close the window; [Appendix B](#) compares the repairs.

P2: a report of the proposer’s own lease is open only during a renewal of that exact lease. The reported ballot must equal the renewal base captured at P1; ownership alone does not qualify a response.

A proposer abandons an attempt (P5) with accept requests in flight and retries under a fresh ballot, and in between a competitor on its first attempt acquires under a lower ballot. The late accept requests overwrite the competitor’s record at every acceptor, since ballot order permits it and overwriting revokes nothing, and the retrying proposer’s prepare then finds its own lease everywhere it looks. If ownership alone makes those responses open, it completes the acquisition while the competitor is still active. TLC finds the execution in 36 states at the full quarantine bound with retry enabled (variant `StaleOwnerOpen.tla`), and no quarantine length prevents it, because the stale accept request can be delayed arbitrarily. Both audited implementations ship the unqualified rule ([Section 11](#), seventh finding).

An acceptor-side rule suffices on its own: an acceptor never replaces a live lease of a different owner, even under a higher ballot, so the stale accept request cannot erase the competitor’s record. It is local and does not depend on proposers classifying their own attempts correctly, but it gives up availability, since a live record blocks a conflicting acquisition until it expires. TLC checks both repairs: the renewal qualifier passes the retry configuration exhaustively ([Section 9](#)), and the acceptor rule, checked with the qualifier dropped, passes the same configuration over 337,917,446 distinct states.

Both executions survive a connection-oriented transport,

Variant	Weakened rule	Violation	Trace
LateTimer	timer starts at prepare-quorum receipt	LeaseExclusivity	27 states
OwnerOnlyRelease	release matches owner, ignores ballot	LeaseExclusivity	36 states
ScalarQuorumCounting	quorum counted by message, redelivering transport	LeaseExclusivity	18 states
StaleOwnerOpen	own lease open without the renewal qualifier, retry enabled	LeaseExclusivity	36 states

Table 1: Counterexample variants: full copies of the specification with exactly one rule weakened, regenerated from it and compared against the checked-in files by `make variant-check`. The target `make counterexamples` requires TLC to find the first three violations on every run; the `StaleOwnerOpen` search is multi-hour and is recorded evidence, validated against the cited trace by `make paper-claims`. The `LateTimer` and `StaleOwnerOpen` configurations use the full quarantine bound.

in which a crash discards the messages addressed to the crashed process; [Appendix C](#) gives that refinement and its verdicts.

9. Checked Models and Tests

The paper’s [accompanying GitHub repository](#) contains the TLA+ specification and its configurations, the deliberately weakened variants of it, the TLAPS proof module, two Python models, the tests, and the recorded output of every run; [Appendix D](#) inventories it. The paper’s negative claims, that some rule cannot be dropped, are verified by modified variants of the original: copies of the specification, `tla/spec/PaxosLease.tla`, each with exactly one rule weakened and nothing else changed. A script generates the variants from the specification, so no variant can drift away from the specification it claims to weaken. TLC must then find the advertised violation in three of them on every run, and the fourth is a recorded run ([Table 1](#)).

The positive claims are exhaustive, but only for the configurations checked, and those are small: no exhaustively checked configuration has more than two proposers, three acceptors or three ballots, and each fixes one duration triple (D_P, D_A, Q) . Inside such a configuration TLC enumerates every reachable state and finds no violation of `LeaseExclusivity`; about larger ones it says nothing, and [Section 14](#) says which of these bounds the result actually rests on. [Appendix D](#) lists them all. They cover the separated-duration boundary pairs of [Section 7](#), explicit retry at the quarantine boundary, and the redelivering transport. Three further configurations test feature interactions, combining renewal, retry, release, and redelivery. The last of these was added after the stale-owner execution showed that features checked in isolation are not features checked together. A further family of configurations projects the rules the two audited implementations actually ship, in particular a retry timeout in place of a stored attempt deadline, and reproduces both machine-found failures under the constants those systems ship ([Section 11](#)).

The four variants of [Table 1](#) correspond one for one to the rules this paper says cannot be dropped. The first, `LateTimer`, is the timer placement of [Section 8](#), whose 27-state trace [Appendix B](#) walks through in full. The

second completes the release story of [Section 4](#). If release matches on owner alone, then a release message delayed past a re-acquisition by the same owner erases the newer lease. The owner releases instance $(p_1, 1)$ and re-acquires as $(p_1, 2)$, the stale release then clears the ballot-2 lease at the acceptors, and a second proposer acquires while p_1 is still active, in a 36-state trace. The third records the deduplication stake of [Section 11](#): with quorums counted by message under a transport that may redeliver, a single acceptor’s doubled response is a quorum, and two proposers become active in 18 states with no crash, no restart, and no clock movement at all. The fourth is the stale-owner execution of [Section 8](#): the base module with P2’s renewal qualifier dropped, whose 36-state two-owner search is the multi-hour recorded run `make counterexamples-staleowner`.

The safety argument of [Section 6](#) rests at several points on inequalities between durations, and those inequalities are proved mechanically, by TLAPS, in `tla/proof/PaxosLeaseProof.tla`: timer containment, the quarantine bound (both forgotten facts reduce to attempt lifetime), monotonicity of the deadline cap used by the finite model, tightness of the quarantine bound, and the clock-rate condition of [Section 3](#).

10. Runnable Demonstration

To aid understanding for practitioners, the accompanying repository contains one more program, `python/demo.py`, a dependency-free, single-file implementation of the algorithm of [Section 4](#). It runs three nodes in one event loop on real monotonic-clock timers, and each node hosts a proposer, an acceptor, and a learner. Every method names the step it implements, P1 through P5, A1, A2 and A4, L1, and the timing rules, so the file reads against the paper. Where this paper shows that a plausible alternative to a rule is unsafe, the comment at that rule says so and points at the section. The program demonstrates normal operation: a node acquires the lease, renews it at half-life, and dies; a survivor takes over once lease and quarantine have run out; and a referee object receives every learner-level ownership claim and asserts throughout that no two nodes are ever owner at once.

11. Audit of an Unformalized Protocol

The protocol of [Section 4](#) existed as a paper and open source implementations for over a decade with no machine-checked model. This section records what went unnoticed as a result, first in the specification and then in the code. The trace walkthroughs and the source-level detail are in [Appendix B](#) and [Appendix C](#).

The previous paper describes the start of the proposer’s timer twice and the two descriptions disagree. Its figure draws the safe rule, the deadline stored when `Prepare` is broadcast, which is the rule this paper specifies as T1; its pseudocode states the quorum-receipt rule that [Section 8](#) shows unsafe; and its prose is consistent with both. The counterexamples are small and need nothing exotic (27 states; 25 with three acceptors and a single crashed acceptor), and [Appendix B](#) gives one in full.

PaxosLease was implemented by this author in two open source systems, Keyspace [14] and later ScalienDB [15]. A protocol author auditing his own fifteen-year-old code against his own checked model is an unusual exercise, so we record the findings in detail. Keyspace’s implementation of the algorithm lives in `src/Framework/PaxosLease/`, and ScalienDB’s in `src/Framework/Replication/PaxosLease/`. The audit reads the final commits of both repositories (a99f24a8 of 2011 and 60978146 of 2013), and produced eight findings; the details are in [Appendix C](#).

1. **Both systems implement the unsafe timer rule of the pseudocode, but an unrelated retry timeout accidentally prevents the two-owner executions.** The rule that shipped is the *unsafe* one of the pseudocode, not the safe one of the figure: `StartProposing()` sets `expireTime = Now() + duration`, so the authority clock starts when the prepare quorum is in hand, exactly the rule that [Section 8](#) shows unsafe. What both contain instead is a mechanism the previous paper never mentions, an attempt-restart timeout of $R = 2$ seconds armed when `Prepare` is broadcast, whose firing re-prepares under a fresh proposal identifier and thereby discards the abandoned attempt’s responses. This is the attempt time bound enforced by control flow rather than stored as data, which [Section 8](#) shows insufficient. Neither codebase presents it as a safety mechanism; it serves as one by accident.
2. **That accidental prevention works only under certain conditions, which the implementations accidentally satisfy.** We built a checked projection of the implementations onto their lease acquisition state machine (`tla/spec/PaxosLeaseImpl.tla`) and checked when the accidental repair works. Under timely timeout dispatch the restart condition is

$$Quarantine \geq \max(D_P, R),$$

checked at its boundary in both coordinates and refuted one unit below in each; the boundary run also

refutes the additive hypothesis $Q \geq R + D_P$. The shipped ordering of the constants passes exhaustively.

3. **The event loop’s handler ordering can produce a two-owner execution.** Neither proposer stores when its attempt began; the R bound holds only because each event-loop iteration runs due timers before socket events. The real assumption is therefore stronger than freedom from suspension: no message handler belonging to an expired attempt ever executes past the deadline before the timeout transition runs, and the loop structure does not enforce that: the poll blocks until the next timer is due, so a socket that becomes ready just before the deadline has its handlers, and any burst behind them, dispatched before the next timer scan. A process suspension beginning after a timer scan and ending inside the poll is the dramatic instance; ordinary event-loop overrun is the everyday one. TLC finds the resulting two-owner execution with the shipped constants in 27 states. The shipped constants are therefore not enough to call the implementations safe.
4. **Quorums are counted by message rather than by acceptor identity, which is safe only because the transport happens not to duplicate.** Both systems count quorum responses without acceptor identity, so one response delivered twice counts as two votes. We found no path that retransmits a response: pending writes are discarded on disconnect, and the inspected TCP connections supply non-duplicating delivery within a connection. The scalar counting therefore rests on an application-level at-most-once assumption that is plausible for these implementations but stated and checked nowhere, and the weakened variant `ScalarQuorumCounting.tla` shows what it protects: under a transport that may re-deliver, TLC finds a two-owner execution in 18 states.
5. **Every lease timer reads the wall clock, and neither system defends both of the directions in which a wall clock is unsafe.** Clock errors are not symmetric between the roles: backward or slow steps are unsafe on a proposer (authority extends) and merely prolong exclusion on an acceptor, while forward or fast steps are unsafe on an acceptor (exclusion shrinks) and harmless on a proposer. Keyspace uses `gettimeofday()` raw and is exposed in both unsafe directions; ScalienDB repairs backward steps well, through a correction thread, but lets forward steps pass uncorrected. Neither system calls the monotonic clocks its platforms provided, `clock_gettime(CLOCK_MONOTONIC)` on Unix and `GetTickCount64` or `QueryPerformanceCounter` on Windows.
6. **The learner is where ownership becomes visible, and its remote expiry rests on an undocumented delivery bound.** Ownership is application-visible only through the learner, and the learner restarts the

countdown from the moment the message arrives rather than from the moment the lease began. In both systems the owner’s own learner installs the absolute expiry the proposer computed, so the two-owner executions above carry through to two simultaneously application-visible masters; but every *remote* learner sets its expiry to arrival time plus the remaining duration minus a 500-millisecond hedge, so the hedge is conservative only while LearnChosen delivery takes less than 500 milliseconds, another undocumented timing assumption.

7. **A proposer treats any lease it owns as an open response even when it is no longer the active owner.** Both proposers treat a reported lease they own as an open response whether or not they are currently the active owner: StartProposing proceeds, with a full fresh duration, whenever the discovered lease owner is the node itself. Section 8 shows the rule unsafe in combination with the retry timeout of the first finding: an accept request from an abandoned attempt can overwrite a competitor’s live lease record and then serve as its sender’s own justification, and TLC finds the resulting two-owner execution in 36 states at the full quarantine bound, with no dispatch delay and no clock error. The execution does not depend on connectionless delivery: under the connection-oriented refinement, in which a crash discards the messages addressed to the crashed process, TLC finds it at the same depth, with the stale accepts sent after the acceptors restart, which a reconnecting writer performs faithfully. The repair is the renewal qualifier that P2 states: an own lease is an open response only for a renewal attempt, and only for the exact lease captured as its base. The composition is checked in the implementation projection itself, not only in the base model: with three ballots, the shipped timely constants, and the connection-lifecycle transport, TLC composes the shipped retry timeout with the shipped own-lease rule into a 35-state two-owner execution (PaxosLeaseImplStaleOwner.cfg). Timer dispatch is timely throughout that execution: it needs no suspension, no dispatch delay, and no clock error.
8. **The activation guard reads the clock twice, and an unsigned subtraction can wrap so that a proposer activates a lease that has already expired.** The Phase 2 handlers read the clock once for the expiry guard and again for the activation-margin subtraction, on `uint64_t` deadlines. If the clock crosses the deadline between the two reads, through a pause between the calls or a forward step, the unsigned subtraction wraps, the margin check passes, and the proposer activates an expired lease; Keyspace additionally broadcasts the wrapped value as the lease duration. The window is narrow, but it is a time-of-check to time-of-use race on the safety-critical comparison, and the repair, one clock read per handler with a compare-

before-subtract, is standard.

12. Lessons

Writing this paper taught the author several lessons, and none of them is specific to PaxosLease. They are enumerated below, in the order in which the work runs.

1. **Algorithms need to be formally specified and checked.** Lamport’s standing advice¹ is to write the algorithm in TLA+ and let the model checker debug it.
2. **The implementation should be derived from the specification.** Both audited systems were written from the pseudocode, and the pseudocode was the one place where the safety-critical rule was stated wrongly, so the diagram that had it right never reached the code.
3. **The implementation must also be re-encoded as a specification and checked.** Software adds networking and messaging code, event handling and control flow, concrete data types for the algorithm’s variables, and other implementation detail, and each of these is an opportunity to introduce a bug into an otherwise correct algorithm. By translating the software implementation back to a specification and then checking it, the risk of shipping unsafe code to production is reduced.
4. **Frontier language models speed up both the derivation and the re-encoding steps.** The time required to write and maintain a specification, its configurations, its weakened variants and its harness has discouraged engineers from using formal methods. Modern frontier models automate away much of this work. Extensive human guidance and review are required throughout the process.

13. Related Work

Leases as time-bounded ownership originate with Gray and Cheriton [3]. Chubby [6] made the lease-plus-Paxos architecture standard practice at scale, and the engineering account of [7] describes how much of the difficulty lies outside the core algorithm. FaTLease [8] solves the same lease-negotiation problem as PaxosLease, but it runs Paxos instances for the lease commands and assumes clock synchrony, while PaxosLease was designed to remove both [1]. The closest relative is Flease [10], from the same

¹In December 2009 the author described PaxosLease in an email to Leslie Lamport, who replied that he did not have time to look at the algorithm and suggested coding it in TLA+ and using the model checker to debug it. This paper is that suggestion, carried out seventeen years later. He was right on both counts: the model checker was the proper instrument, and the algorithm did need debugging.

group as FaTLease: decentralized lease coordination without stable storage for the lease state, built on a round-based register.

Raft [4] structures leadership through terms and elections, but its safety, like that of Paxos, does not depend on real-time exclusivity. Production Raft systems that serve reads from the leader without a log round trip add a leader lease, and they inherit exactly the containment and clock-rate obligations of Section 3. Paxos Quorum Leases [12] generalize the lease holder from a single leader to a quorum of replicas that may serve local reads. They are a read-performance mechanism layered on Paxos rather than a diskless leadership protocol, but their correctness rests on the same real-time containment argument, applied at each lease-holding replica.

On the verification side, Chand, Liu, and Stoller [5] give a full TLAPS proof of Multi-Paxos, and Grove [11] is the modern benchmark for mechanized lease reasoning, verifying time-based leases together with crash recovery, reconfiguration, thread-level concurrency, and unreliable networks in an executable key-value store. Lamport’s Paxos papers [2, 9] set the specification style, and the trace-validation work of Cirstea et al. [13] shows how the remaining gap between such models and an implementation can be closed mechanically.

14. Limitations

Below we list the paper’s most significant limitations:

1. PaxosLease assumes non-Byzantine processes that follow the protocol and stop acting when their leases expire.
2. Only majority quorum systems were checked. The argument of Section 6 uses nothing but quorum intersection, which every quorum system provides, but no non-majority configuration was run through TLC.
3. The acceptor set is fixed. Reconfiguration requires preserving quorum intersection across configurations, or else waiting until leases from the old configuration can no longer matter.
4. The TLC runs are finite-state checks, no configuration with more than two proposers, three acceptors or three ballots was checked, and each configuration also caps the number of messages in flight. They do not prove the protocol for all quorum systems or all cluster sizes. The acceptor bound is the least binding of these, since the argument of Section 6 uses only quorum intersection and so holds for majorities of any size; the proposer and ballot bounds are where the result genuinely rests on finite checking. The ballot bound also limits retry chains, since a four-ballot configuration, admitting two consecutive retries beside a

competitor, was terminated at a time cap without a verdict.

5. The clock model assumes monotonic elapsed-time measurement with bounded rate error. A process that is suspended, paused by its hypervisor, or live-migrated stops running while real time keeps passing, so its lease can expire before it next looks at its timer, and it may go on acting as owner when it resumes.
6. Only the timing arithmetic is mechanically proved. The exclusivity theorem rests on three things together, none of them a proof of it: the argument of Section 6, those arithmetic lemmas, and finite model checking.
7. The implementation audit of Section 11 is a reading of the sources against the checklist of the model, backed by a checked projection of the implemented protocol and an executable abstraction of its node. We did not construct the described executions against running binaries, and the quantitative claims about margins assume the shipped compile-time constants.

15. Conclusion

PaxosLease grants time-bounded exclusive ownership from a quorum of acceptors that keep no durable lease state. Its standard use is to elect a Multi-Paxos leader, which runs Phase 1 once per epoch and then appends with one round trip and one durable write per value for as long as it renews the lease. Safety rests on quorum certificates whose acceptor exclusions outlast the proposer’s authority, on ballots that order competing attempts, on a quarantine of one attempt duration that makes forgotten state safe across an acceptor restart, and on release and renewal rules that keep a proposer’s own stale messages from being read as evidence of its authority. Each clause is paired here with a machine-found execution showing what happens without it.

Lamport’s advice, given to the author in 2009, taken seventeen years later, was right on both counts: specifying and checking the model was the right next step, and both the algorithm’s description in the original paper and the implementations needed debugging. The work also suggests a way of building such systems:

1. specify the algorithm formally and check it;
2. derive the implementation from that specification;
3. encode the implementation, with its transport, its event loop and its data types, as a specification and check it in turn.

Frontier language models make both encodings fast enough to be routine, with human direction and review throughout. In line with that learning, large language models assisted in constructing and revising these models and programs, under the direction and review of the author.

Appendix A. Checked Invariants

Appendix A.1. Standalone PaxosLease Invariants

TypeOK: Every variable remains in its declared finite domain: times are bounded, messages have one of the modeled shapes, acceptor state maps acceptors to lease records, proposer phases are among the modeled phases, and response sets are sets of acceptor identities. This is the well-formedness condition needed before the other predicates have their intended meaning.

AcceptedCoherence: An acceptor’s accepted value is either `NoLease` or a lease whose owner, ballot, and deadline lie in the modeled domains. This rules out malformed accepted state and ensures that later prepare responses can be interpreted as lease reports.

ActiveImpliesUnexpired: Every active proposer has a local deadline strictly greater than the current model time. This captures the rule that a proposer stops acting when its own lease interval expires.

LeaseExclusivity: The set of active proposers has cardinality at most one. This is the main safety property. It is checked directly by TLC, checked by the Python simulator after every operation, and violated on purpose by every counterexample variant.

ActivationHasQuorum: An active proposer has a quorum certificate. A proposer cannot become active merely because it sent requests or received a non-quorum set of responses.

TimerContainment: For every acceptor in an active proposer’s certificate, the proposer’s deadline comes no later than the acceptor deadline recorded in the certificate. The certificate variables are history variables (Section 3), so the invariant is a statement checked by the model, not a comparison performed by the protocol.

QuarantinePreventsParticipation: An acceptor in quarantine has no promised ballot and no accepted lease. Together with the transition rules, this means that a restarted acceptor cannot vote while forgotten lease state could still matter.

ActiveBallotWellFormed: An active proposer’s active ballot is one of the modeled ballots. This is a bookkeeping invariant for the certificate machinery. The substantive renewal property, that a failed renewal does not extend authority, is a transition rule exercised by the renew/release configuration and by the scenario tests.

Appendix A.2. PaxosLease+Paxos Invariants

ChosenValueWellFormed: The abstract composition model represents each slot by a single chosen value or by `NoValue`, and the transition relation assigns a chosen value only when the slot is empty. Agreement is therefore enforced by construction in this abstraction, the named invariant checks domain membership, and the substantive checks are the admission invariants below. We state this explicitly to avoid overclaiming, since the composition model checks the lease/Paxos boundary, not Paxos itself.

ReadyImpliesActive: A ready leader holds a live lease. This is the admission side of the composition, since a process without lease authority cannot be ready to accept client log work.

ReadyLeaderUniqueness: At most one proposer is in the ready state. This is stronger than Paxos requires for agreement, but it is the real-time leadership property that the lease layer supplies.

RecoveryPrecedesAdmission: A proposer cannot apply or admit log work before it has completed recovery. This is the modeled boundary between the lease protocol and Paxos recovery.

ReadyImpliesRecovered: A ready leader has completed recovery within its current lease epoch, since acquiring a lease clears the recovery mark. This fences the fast path per epoch: no leader appends on the strength of a previous epoch’s recovery.

AppliedImpliesChosen: Every slot applied by any proposer has a chosen value, so the leader-admission abstraction cannot invent applied log entries that Paxos has not chosen. **Log Prefix Consistency** is the two-proposer restatement of the same fact, implied by it and retained as a separate named check for readability.

Appendix A.3. TLAPS Obligations

TimerContainmentArithmetic: If the proposer starts its timer no later than each certificate acceptor starts its exclusion, and its duration is no longer, then the proposer’s authority interval is contained in the acceptor’s exclusion interval.

QuarantineCoversForgottenFacts: Everything an acceptor can forget, whether a promise or an accepted lease, belongs to a proposer attempt whose `Prepare` was sent no later than the crash, and under the prepare-time timer rule that attempt is unusable D_P later. Quarantine at least D_P therefore outlasts every attempt that any forgotten fact could still support. The obligation has no analogue under the step-3 rule alone; with a retry bound the analogue is the next obligation.

ImplQuarantineCoversForgottenFacts: The analogue for the implemented protocol under timely timeout dispatch: a forgotten promise anchors at a `Prepare` and is unusable R later, a forgotten accepted lease anchors at a prepare-quorum receipt and is unusable D_P later, and both anchors precede the crash, so quarantine at least $\max(D_P, R)$ covers both cases.

CapMonotone: The deadline cap that keeps the finite model bounded is monotone, so capping preserves containment.

QuarantineBoundTight: Any quarantine strictly below the proposer duration leaves an elapsed interval inside a forgotten attempt’s lifetime but outside quarantine. TLC turns this arithmetic gap into the concrete two-owner traces of Section 7.

DriftContainmentArithmetic: If the proposer’s clock runs at rate at least r_{min} , the acceptor’s at most r_{max} , and the durations satisfy $D_P \cdot r_{max} \leq D_A \cdot r_{min}$, then real-time containment holds. This is the cross-multiplied form of the inequality of Section 3, and the same arithmetic with the quarantine in place of D_A gives the real-time quarantine bound.

Appendix B. Inconsistency in the Original Statement

Figure 2 of [1] places “start timer” on the proposer’s lane before the prepare requests leave. The pseudocode instead starts it in step 3, `Proposer : OnPrepareResponse`, when a quorum of empty prepare responses has arrived, immediately before sending propose requests. The proof’s prose says that the proposer starts its timer before sending propose requests, which is consistent with both placements.

Under the pseudocode rule, an attempt has no deadline between sending `Prepare` and processing its prepare quorum. The original restart quarantine of the maximal lease time M outlasts forgotten exclusions and attempts whose timers have started. It cannot outlast an attempt whose timer may start arbitrarily late. No finite quarantine repairs that rule.

`tla/counterexamples/LateTimer.tla` changes only timer placement: `SendAccept` sets the pending deadline instead of `StartAcquire`. With two proposers, two acceptors, and $D_P = D_A = \text{Quarantine} = 2$, TLC finds a 27-state violation of `LeaseExclusivity`. The execution proceeds as follows.

1. At $t = 0$, p_1 and p_2 send `Prepare` with ballots 1 and 2. Both acceptors promise in ballot order, and both proposers receive empty promise quorums. Neither proposer has started its timer.
2. Both acceptors crash, forgetting their promises. They restart and remain in quarantine until $t = 2$.
3. At $t = 1$, both proposers process their prepare quorums. Each starts its timer with deadline $t = 3$ and sends `Accept`.
4. At $t = 2$, quarantine ends. The acceptors accept ballot 1, then overwrite it with ballot 2; the crash erased the promises that would have rejected ballot 1.
5. Both proposers receive accepted quorums and activate. At $t = 2$, both own the lease until $t = 3$.

Every acceptor served its full quarantine. The execution needs no message loss or clock error. The original invariance argument omits acceptor state loss; its restart wait assumes that every attempt using forgotten state already has a bounded lifetime.

In implementation terms, ordinary scheduling delay separates receipt of the last prepare response from sending the propose requests. A

garbage-collection pause or a preempted virtual machine, together with acceptor crash and restart, supplies the whole schedule. The proposer resumes with responses that remain acceptable under the pseudocode despite the loss of the promises they report.

Rule T1 repairs this by storing the deadline when `Prepare` is sent and checking it before sending accepts and activating. Unprocessed promises expire with the attempt, so quarantine D_P suffices (Section 7). Network and processing delay reduce the usable lease interval. The `late-promise-reuse` Python witness reproduces the unsafe schedule; its paired test uses the prepare-time rule, rejects the expired promise quorums, and sends no `Accept`.

Durable promises also prevent the execution, at the cost of a synchronous disk write on the prepare path. An expiry token on prepare responses applies the same time bound as the first repair. Fleas’s clock-derived ballots use stronger clock assumptions (Section 13). A separate retry deadline can bound the attempt if stored and checked in every response handler. The audited implementations use a scheduled retry callback without those checks; delayed dispatch defeats it (Appendix C). For a separate retry bound R , quarantine must cover both the retry interval and the authority interval. The implementation projection checks $Q \geq \max(D_P, R)$ under timely dispatch. Storing and enforcing the retry deadline removes that dependence on callback ordering.

With three acceptors and majority quorums of two, TLC finds a 25-state two-owner trace under proposer and acceptor symmetry reduction. Only the acceptor at the quorum intersection crashes; it forgets its promise, serves the full quarantine, and participates in both accept quorums. The failure therefore also occurs when one acceptor remains available to each proposer throughout. Separately, quarantine one unit below D_P yields a 24-state trace with three acceptors. The repository records these searches and the fail-fast target validates their traces and counts; `make paper-evidence-full` reruns them.

Appendix C. Audit Detail: Keyspace and ScalienDB

This appendix records the source-level evidence behind the eight findings of Section 11. The audit reads Keyspace at commit a99f24a8 (2011) and ScalienDB at commit 60978146 (2013), the final commits of both repositories.

Appendix C.1. The timer rule and the retry timeout

In both systems `StartProposing()` sets `expireTime = Now() + duration` (Keyspace `PLeaseProposer.cpp`, ScalienDB `PaxosLeaseProposer.cpp`). The attempt-restart timeout is `ACQUIRELEASE_TIMEOUT`, 2 seconds in both, armed by `StartPreparing()`; when it fires, `OnAcquireLeaseTimeout()` calls `StartPreparing()` again under a fresh proposal identifier. The maximal lease time is `MAX_LEASE_TIME`, 7 seconds in Keyspace and 3 in ScalienDB, and both the lease duration and the startup quarantine are set to it.

`OnProposeResponse` contains the unsigned-arithmetic race of the eighth finding. It tests `state.expireTime < Now()` and returns if the lease has expired, then, with a separate clock read, tests `state.expireTime - Now() > 500` before activating; `expireTime` is `uint64_t` (`PLeaseState.h`). A clock that crosses the deadline between the two reads wraps the subtraction to nearly 2^{64} , which passes the margin test, and Keyspace then passes `state.expireTime - Now()`, a third read, to `LearnChosen` as the lease duration. The `unsigned-expiry-underflow` witness records the arithmetic.

Appendix C.2. The implementation projection

`tla/spec/PaxosLeaseImpl.tla` differs from the base specification in exactly the two audited rules: `pendingDeadline` is set in `SendAccept` rather than `StartAcquire`, and an attempt deadline of R units, set at `StartAcquire`, abandons the ballot when it fires. The Phase 2 delivery action has no attempt-deadline guard, matching `OnProposeResponse`. The constant `TimelyDispatch` selects the timeout semantics. Under timely dispatch, advancing time abandons every overdue attempt before anything else happens, which models an event loop that is never paused. Under delayed dispatch, expiry only enables

an abandonment action that any amount of message processing may postpone, which is what a suspension produces.

Appendix D lists the module’s timer and quarantine configurations, in (D_P, R, Q) , with their state counts. Since the boundary passes and one unit below it fails, the condition the shipped constants must satisfy is $R \leq M$, not $R < M$, and both implementations satisfy it strictly. The coverage lemma behind the bound is the TLAPS obligation `ImplQuarantineCoversForgottenFacts` (Appendix A).

The two-acceptor delayed-dispatch trace crashes both acceptors, and in the deployed systems a node crash restarts the colocated proposer and bumps the durable restart counter in its proposal identifiers, so in a two-node deployment that trace would change both proposers’ ballot ordering. `PaxosLeaseImplDelayedColocated.cfg` therefore uses three acceptors and restricts `CrashableAcceptors` to the single acceptor in the intersection of the two proposers’ quorums. No proposer-hosting node then crashes, no restart counter moves, and the node-identity tiebreak between equal attempt counters is legitimate, so its 25-state trace is valid for the colocated deployment without modeling restart counters; it is the recorded execution behind the schedule below.

Appendix C.3. The violating schedule

A process that merely resumes after a long pause fires the overdue retry at the next timer scan and discards the stale responses, so the schedule pauses inside one loop iteration, between `RunTimers()` and the return of `IOProcessor::Poll()` (Section 11, third finding). It crashes exactly one node, the acceptor in the intersection of the two quorums, which hosts neither active proposer.

1. Node 2’s proposer completes its prepare round against acceptors 1 and 2, starts its authority interval, and sends its accept requests, which are delayed.
2. Node 2’s loop performs the last timer scan before the retry deadline and blocks in the poll. The suspension begins there and outlasts node 1’s quarantine.
3. Node 1 alone crashes after promising, restarts, and serves out its full quarantine.
4. Node 0’s proposer completes an entire fresh acquisition against acceptors 0 and 1, and its learner installs the absolute expiry: node 0 is the application-visible master.
5. The stale accept requests are accepted at node 1, overwriting the record of the fresh lease without revoking its holder’s authority, and, on resume, at node 2’s own acceptor. Node 2’s stale ballot exceeds node 0’s fresh one because the attempt counters are equal, neither node restarted, and the tie breaks on node identity, exactly the counter-major layout.
6. Node 2’s queued responses are then dispatched before the overdue timeout callback, the handler finds the lease expiry unreached with more than its 500-millisecond margin remaining, the proposer activates, and its own learner installs the absolute expiry. Nodes 0 and 2 are now simultaneously application-visible masters.

`python/paxoslease/event_loop.py` replays this schedule against an executable abstraction of the shipped node, with proposer, acceptor, and learner colocated, as the `suspended-event-loop` witness, asserted at `IsLeaseOwner()` level; a paired run shows the same schedule refused without the suspension. Exceeding the 500-millisecond margin requires only a virtual-machine pause, a swap stall, a `SIGSTOP`, or a long enough handler burst.

Appendix C.4. Quorum counting and the transport

Keyspace’s proposer counts scalars (`numReceived++`, `numAccepted++`), and ScalienDB’s vote object takes a node identifier but uses it only for a membership test before incrementing a scalar (`MajorityQuorum.cpp`). Pending writes are discarded on disconnect in `TCPConn::Close` (Keyspace) and `TCPConnection::Close` (ScalienDB). A proof of at-most-once delivery across connection replacement would also have to exclude simultaneous old and new connections to one peer, regenerated responses on reconnect, and handler re-entry, which we have not done, and Keyspace’s tree contains an unused UDP transport. The implementation contract states the rule disjunctively: count by identity, or document and preserve at-most-once delivery per response.

Configuration	(D_P, D_A, Q)	Purpose	Distinct states	Result
Base	(2, 2, 2)	acquisition races, two proposers, three acceptors	573,975	pass
Renew/release	(2, 2, 2)	renewal and exact-instance release	7,717	pass
Crash/restart	(2, 2, 2)	two proposers race across crash+restart	10,059,404	pass
Drift	(1, 2, 2)	drift-adjusted (shorter) proposer duration	19,656	pass
Quarantine = D_P	(1, 2, 1)	boundary: quarantine below acceptor exclusion	20,447,948	pass
Quarantine = D_P	(2, 3, 2)	boundary at a second duration pair	9,867,548	pass
Redeliver	(2, 2, 2)	redelivering transport, identity counting, no crash	1,857,563	pass
Redeliver+Crash	(2, 2, 2)	redelivering transport across acceptor crash and restart	494,875,842	pass
Retry	(2, 2, 2)	explicit abandon and retry, three ballots, crash	251,904,392	pass
Renew+Retry	(2, 2, 2)	renewal and abandon together, crash	280,165,306	pass
Retry+Redeliver	(2, 2, 2)	abandoned attempts under redelivery	53,723,103	pass
Renew+Release+Retry	(2, 2, 2)	stale Phase 2 traffic around renewal and release	204,050	pass
Lease+Paxos	n/a	abstract composition barrier	2,095	pass
Unsafe quarantine	(1, 2, 0)	quarantine one unit below D_P	not exhaustive	violated (25-state trace)
Unsafe quarantine	(2, 3, 1)	quarantine one unit below D_P, D_A larger	not exhaustive	violated (26-state trace)

Table 2: TLC checks of the unmodified specification. Passing counts are exhaustive for the finite configurations, subject to the per-configuration bound on in-flight messages. Violated runs stop at the counterexample; their trace lengths are minimum-depth under breadth-first search and stable across runs, while the number of states a parallel search happens to explore before finding the violation is not, so it is not cited. The two boundary rows are the separated-duration experiments that establish $Quarantine \geq D_P$ as the tight bound in the checked model. The Retry and Renew+Retry rows, at a quarter-billion states each, are multi-hour recorded runs; the rest run in the fail-fast pipeline.

Rule	Base module	Variant/projection	Python witness	Source audit
Attempt time bound (P1, T1)	StartAcquire stores t_d	LatTimer	simulator, timer witnesses	unsafe rule shipped
Retry time bound (P5)	AbandonAttempt, Retry cfg	impl projection (R)	n/a	undocumented $R \leq M$
Exact-instance release (A3)	DeliverRelease	OwnerOnlyRelease	release witness	no release path shipped
Restart quarantine (A4, T3)	RestartAcceptor	unsafe-quarantine cfgs	quarantine witness	full- M quarantine
Identity counting (P2)	response sets, Redeliver	ScalarQuorumCounting	duplicate witness	scalar counters, TCP accident
Own-lease-open qualifier (P2)	DeliverPromise	StaleOwnerOpen	stale-owner-open witness	unqualified rule shipped
Learner deadlines (L1)	not modeled	n/a	event-loop witness	remote re-anchoring
Clock discipline	not modeled	n/a	underflow witness	double clock read
Leader lifecycle (M1–M5)	composition model	n/a	leased_paxos.py, tests	recovery conflation

Table 3: Each rule of Sections 4 and 5 mapped to the models and tests that check it. “Not modeled” marks the implementation-layer rules that the base transition system deliberately abstracts away; their evidence is executable witnesses and the audit of Appendix C.

Appendix C.5. The learner path

Application code reads `IsLeaseOwner()` in `PLearnLearner.cpp` and `PaxosLeaseLearner.cpp`, which the `LearnChosen` broadcast populates at every node, the sender included. The owner’s learner installs `msg.localExpireTime`, the absolute expiry the proposer computed; every other learner installs `Now() + msg.duration - 500`. The source comments call the subtraction a conservative estimate, which it is while delivery takes less than 500 milliseconds; a `LearnChosen` delayed by $\Delta > 500$ ms leaves every remote learner believing in the owner’s authority $\Delta - 500$ ms past its real end. Remote learners do not act as owner, but master lookups and election suppression read this state.

Appendix C.6. Clock call paths

In `Keyspace`, `Now()` is `GetMilliTimestamp()`, which is `gettimeofday()` raw, with no clamp and no correction (`Platform.cpp`; the `unsafe-clock-source` witness). In `ScalienDB`, every lease-layer call reaches the `Now()` of `Time.cpp`: `gettimeofday()` plus a global correction offset, with a per-thread monotonic clamp. A clock thread started unconditionally at boot (`Main.cpp`) samples the clock every 5 milliseconds and, on a backward step, permanently raises the offset so that corrected time resumes at its previous value plus one resolution step, so elapsed intervals measured across the step remain approximately correct. Forward steps pass through uncorrected in both systems. `ScalienDB`’s `Windows` path in `Time.cpp` builds an elapsed-time scheme on `timeGetTime()`, milliseconds since boot, and never runs it: the enclosing `gettimeofday()` opens with `return gettimeofday_win(tv, NULL)`; and everything below that return is unreachable.

Appendix C.7. Connection-lifecycle transport

Under `CrashDropsIncoming`, messages addressed to a process are discarded when it crashes and again when it restarts, so every delivered

message was sent after its addressee’s most recent restart; messages a crashed process already sent survive. The verdicts do not change. The late-timer execution survives unmodified at 27 states, because its stale messages are promises, sent by the acceptors before they crash. The stale-owner execution survives at 36 states: the acceptors crash and restart first, and the abandoned attempt’s accept requests are sent afterward, on fresh connections, which a reconnecting writer performs faithfully. In the implementation projection, the colocation-valid configuration under this transport (`PaxosLeaseImplDelayedCollocatedTcp.cfg`) finds a 25-state two-owner execution, and the stale-owner projection of the seventh finding runs under the same transport. Connection teardown masks none of these executions; the transport property safety rests on is the at-most-once delivery of the fourth finding.

Appendix C.8. What the implementations get right

Both systems quarantine at the full M : `Keyspace` boots with its lease transport reader stopped for `MAX_LEASE_TIME`, and `ScalienDB` drops every lease message while its startup timeout is active. Both persist the restart counter of the footnote of [1], `Keyspace` as `@@restartCounter` in its table store and `ScalienDB` as `runID` committed at boot, and splice it into the proposal identifier below the attempt counter. Both systems raise their proposal counter above any identifier observed in incoming requests, the catch-up rule of T4. Acceptors in both systems lazily expire accepted state before answering a prepare. Neither system implements release.

Appendix D. Repository Inventory

This appendix inventories the evidence behind Section 9: every TLC configuration of the unmodified specification with its state count and verdict, the mapping from each rule to the model or test that checks it, the implementation-projection configurations, and the Python models.

Table 2 lists the configurations of `tla/spec/PaxosLease.tla`. The two boundary rows are the separated-duration pairs of Section 7, and the three rows that combine renewal, retry, release and redelivery are the interaction configurations of Section 9.

Every configuration bounds the number of in-flight messages with the constant `MaxNetwork`, so a passing count is exhaustive for the constrained state graph (Section 14). A violation found under the bound is a legal trace of the unbounded model, so the bound does not weaken the violated rows. For the passing rows we measured its effect. The two-acceptor crash and boundary configurations have identical state counts at bounds 4 and 5, so the bound is never reached in them. Raising the bound on the three-acceptor Base configuration enlarges the state graph without changing its verdict, the Retry configuration at bound 5 passes over 465,991,204 distinct states, and the violating executions of this paper need at most four concurrent messages.

Not every rule of Sections 4 and 5 lives in the base module, and Table 3 states which model or test checks which rule. The proposer, acceptor and timing rules are in the base module, checked in every configuration. The learner rule L1 and the clock discipline concern how an implementation surfaces activation and expiry, which the base module abstracts away, so their evidence is the executable witnesses and the source audit of Appendix C. The leader lifecycle M1 through M5 is checked by the TLA+ composition model, which checks the admission barrier between the two layers, and by the Python model, which runs the recovery, repair and append rounds against ballot-checking durable acceptors.

The implementation projection `tla/spec/PaxosLeaseImpl.tla` of Section 11 has seven timer and quarantine configurations, written (D_P, R, Q) , beside three transport and stale-owner projections. Three pass under timely timeout dispatch: $(D_P, R, Q) = (2, 2, 2)$ over 37,160,904 distinct states, $(1, 2, 2)$ over 37,476,080, and the shipped ordering $(2, 1, 2)$ over 18,160,464. Four violate `LeaseExclusivity`: quarantine one unit below the boundary, at $(2, 2, 1)$ and $(1, 2, 1)$, each in 26 states; the shipped constants under delayed dispatch with two acceptors, in 27 states; and the colocation-valid form with three acceptors, in which only the quorum-intersection acceptor may crash, in 25 states. A further projection combines the shipped retry timeout with the unqualified own-lease rule under timely dispatch and the connection-lifecycle transport, and finds a 35-state two-owner execution (Section 11, seventh finding).

The Python layer holds two deterministic models with virtual time, a lease-only model and a composed lease-plus-Paxos model. They encode the rules a second time, independently of the TLA+, and assert the safety invariants after every step of every schedule they run. TLC searches all schedules within a configuration; the Python models replay specific ones. Fifteen structured witnesses keep every known failure executable, and each must fail with its specific expected message, so an unrelated failure does not satisfy its test.

The four mechanically generated variants of Table 1 each weaken one rule, and TLC finds an exclusivity violation in each; those counterexamples are the stronger class, since there the model checker finds the violating schedule itself. TLAPS proves the six arithmetic obligations listed in Appendix A. The multi-hour searches, the Retry and Renew+Retry rows, the three-acceptor counterexamples and the colocation-valid search, are recorded runs; the fail-fast target validates their traces and state counts against the numbers this paper cites, and `make paper-evidence-full` re-runs them.

Appendix E. How This Paper Was Written

Large language models produced much of the models, configurations, Python programs, source audit, and some of the prose, under the author’s

direction and review. The author selected the questions, reviewed the generated work, and decided which claims to publish. Mechanical checks tested the formal artifacts; prose arguments and source-level claims still required human review.

Every generated formal artifact was independently checkable. TLA+ parsing checked syntax, TLC checked invariants over each configured state graph, and TLAPS checked the arithmetic proof obligations. Weakened variants had to produce their expected counterexamples, and regression targets checked that each witness failed for the advertised reason.

Assistance reduced the recurring work of maintaining the specification, configurations, counterexamples, Python models, and verification pipeline. Revisions were cheap enough that rerunning the checks remained practical after each change, making re-checking habitual. The source audit also benefited from assistance in tracing two old implementations against the model’s rules. Its findings required review of call paths, data types, and event ordering; passing model checks could not establish that the production code implemented those paths as described.

References

- [1] M. Trencsényi, A. Gazsó, and H. Reinhardt. “PaxosLease: Diskless Paxos for Leases.” arXiv:1209.4187, 2012. <https://arxiv.org/abs/1209.4187>
- [2] L. Lamport. “Paxos Made Simple.” *ACM SIGACT News*, 32(4), 2001.
- [3] C. Gray and D. Cheriton. “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency.” *SOSP*, 1989.
- [4] D. Ongaro and J. Ousterhout. “In Search of an Understandable Consensus Algorithm.” *USENIX ATC*, 2014.
- [5] S. Chand, Y. Liu, and S. Stoller. “Formal Verification of Multi-Paxos for Distributed Consensus.” *FM*, 2016.
- [6] M. Burrows. “The Chubby Lock Service for Loosely-Coupled Distributed Systems.” *OSDI*, 2006.
- [7] T. Chandra, R. Griesemer, and J. Redstone. “Paxos Made Live: An Engineering Perspective.” *PODC*, 2007.
- [8] F. Hupfeld, B. Kolbeck, J. Stender, M. Höggqvist, T. Cortes, J. Martí, and J. Malo. “FaTLease: Scalable Fault-Tolerant Lease Negotiation with Paxos.” *HPDC*, 2008.
- [9] L. Lamport. “The Part-Time Parliament.” *ACM Transactions on Computer Systems*, 16(2), 1998.
- [10] B. Kolbeck, M. Höggqvist, J. Stender, and F. Hupfeld. “Flease: Lease Coordination without a Lock Server.” *IPDPS*, 2011.
- [11] U. Sharma, R. Jung, J. Tassarotti, M. F. Kaashoek, and N. Zeldovich. “Grove: a Separation-Logic Library for Verifying Distributed Systems.” *SOSP*, 2023.
- [12] I. Moraru, D. G. Andersen, and M. Kaminsky. “Paxos Quorum Leases: Fast Reads Without Sacrificing Writes.” *SoCC*, 2014.
- [13] H. Cirstea, M. A. Kuppe, B. Loillier, and S. Merz. “Validating Traces of Distributed Programs Against TLA+ Specifications.” arXiv:2404.16075, 2024.
- [14] M. Trencsényi and A. Gazsó. “Keyspace: A Consistently Replicated, Highly-Available Key-Value Store.” arXiv:1209.3913, 2012. <https://arxiv.org/abs/1209.3913>
- [15] M. Trencsényi and A. Gazsó. “ScalienDB: Designing and Implementing a Distributed Database using Paxos.” arXiv:1302.3860, 2013. <https://arxiv.org/abs/1302.3860>